

ASSIGNMENT NO:1

Q. To find average of marks of 5 subjects.

```
Sub1 = int(input("Enter first subject marks: "))
Sub2 = int(input("Enter second subject marks: "))
Sub3 = int(input("Enter third subject marks: "))
Sub4 = int(input("Enter fourth subject marks: "))
Sub5 = int(input("Enter fifth subject marks: "))
avg = (Sub1 + Sub2 + Sub3 + Sub4 + Sub5) / 5;
print("Average Marks =", avg)
```

Output:

```
C:\Users\Admin\PycharmProjects\Python_Lab\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Python_Lab\Avg.py
Enter first subject marks: 88
Enter second subject marks: 86
Enter third subject marks: 90
Enter fourth subject marks: 84
Enter fifth subject marks: 83
Average Marks = 86.2

Process finished with exit code 0
```

ASSIGNMENT NO:2

Q.To find sum of 1st N prime numbers.

```
def is_prime(num):
    if num <= 1:
        return False
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False
    return True

def sum_of_primes(n):
    prime_count = 0
    num = 2
    sum_primes = 0

    while prime_count < n:
        if is_prime(num):
            sum_primes += num
            prime_count += 1
        num += 1

    return sum_primes

n = int(input("Enter the value of n: "))
result = sum_of_primes(n)
print(f"The sum of the first {n} prime numbers is: {result}")
```

Output :

```
C:\Users\Admin\PycharmProjects\Python_Lab\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Python_Lab\Prime.py
Enter the value of n: 13
The sum of the first 13 prime numbers is: 238

Process finished with exit code 0
|
```

ASSIGNMENT NO:3

Q. Write a program

a] To perform linear search

```
def linear_Search(list1, n, Ele):  
    # Searching list1 sequentially  
    for i in range(0, n):  
        if (list1[i] == Ele):  
            return i  
    return -1  
  
list1 = [12, 3, 15, 24, 7, 9]  
Ele = 24  
n = len(list1)  
res = linear_Search(list1, n, Ele)  
if (res == -1):  
    print("Element not found")  
else:  
    print("Element found at index: ", res)
```

Output :

```
C:\Users\Admin\PycharmProjects\Python_Lab\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Python_Lab\Linear_search.py  
Element found at index:  2  
  
Process finished with exit code 0
```

B] To perform binary search

```
def binary_search(list1, n):  
    low = 0  
    high = len(list1) - 1  
    mid = 0  
    while low <= high:  
        mid = (high + low) // 2  
        if list1[mid] < n:  
            low = mid + 1  
        elif list1[mid] > n:  
            high = mid - 1  
        else:  
            return mid  
    return -1  
  
list1 = [12, 24, 32, 39, 45, 50, 54]  
n = 45  
result = binary_search(list1, n)  
if result != -1:  
    print("Element is present at index", str(result))  
else:  
    print("Element is not present in list1")
```

Output:

```
C:\Users\Admin\PycharmProjects\Python_Lab\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Python_Lab\Binary_search.py  
Element is present at index 5  
  
Process finished with exit code 0
```

ASSIGNMENT NO:4

Q. To find factorial of given number.

```
num = int(input("Enter a number: "))  
fact = 1  
if num < 0:  
    print(" Factorial does not exist for negative numbers")  
elif num == 0:  
    print("The fact of 0 is 1")  
else:  
    for i in range(1,num + 1):  
        fact = fact*i  
    print("The factorial of",num,"is",fact)
```

Output :

```
C:\Users\Admin\PycharmProjects\Python_Lab\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Python_Lab\fact.py  
Enter a number: 5  
The factorial of 5 is 120  
  
Process finished with exit code 0  
|
```

ASSIGNMENT NO:5

Q. To find fibonnaci series of given number.

```
n_terms = int(input("Enter number to find fibonacci sequence : "))
n_1 = 0
n_2 = 1
count = 0
if n_terms <= 0:
    print("Please enter a positive integer, the given number is not valid")
elif n_terms == 1:
    print("The Fibonacci sequence of the numbers up to", n_terms, ": ")
    print(n_1)
else:
    print("The fibonacci sequence of the numbers is ")
    while count < n_terms:
        print(n_1)
        nth = n_1 + n_2
        n_1 = n_2
        n_2 = nth
        count += 1
```

Output:

```
C:\Users\Admin\PycharmProjects\Python_Lab\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Python_Lab\fibonnaci.py
Enter number to find fibonacci sequence : 7
The fibonacci sequence of the numbers is
0
1
1
2
3
5
8

Process finished with exit code 0
```

ASSIGNMENT NO:6

Q. To perform sorting for given list using quick sort.

```
def quicksort(arr, low, high):
    if low < high:
        p = partition(arr, low, high)
        quicksort(arr, low, p - 1)
        quicksort(arr, p + 1, high)
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
arr = [10, 27, 8, 19, 1, 15]
n = len(arr)
quicksort(arr, 0, n - 1)
print("Sorted array:", arr)
```

Output :

```
C:\Users\Admin\PycharmProjects\Python_Lab\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Python_Lab\quick.py
Sorted array: [10, 11, 15, 18, 19, 27]

Process finished with exit code 0
```

ASSIGNMENT NO:7

Q. To perform sorting for given list using merge sort.

```
def merge(arr, l, m, r):  
    n1 = m - l + 1  
    n2 = r - m  
    L = [0] * (n1)  
    R = [0] * (n2)  
    for i in range(0, n1):  
        L[i] = arr[l + i]  
    for j in range(0, n2):  
        R[j] = arr[m + 1 + j]  
    i = 0 # Initial index of first subarray  
    j = 0 # Initial index of second subarray  
    k = l # Initial index of merged subarray  
    while i < n1 and j < n2:  
        if L[i] <= R[j]:  
            arr[k] = L[i]  
            i += 1  
        else:  
            arr[k] = R[j]  
            j += 1  
            k += 1  
    while i < n1:  
        arr[k] = L[i]  
        i += 1  
        k += 1  
    while j < n2:
```

```
arr[k] = R[j]
j += 1
k += 1
def mergeSort(arr, l, r):
    if l < r:
        m = l+(r-l)//2
        mergeSort(arr, l, m)
        mergeSort(arr, m+1, r)
        merge(arr, l, m, r)
arr = [9, 11, 20, 5, 16, 7]
n = len(arr)
print("Given array :")
for i in range(n):
    print("%d" % arr[i],end=" ")
mergeSort(arr, 0, n-1)
print("\n\nSorted array :")
for i in range(n):
    print("%d" % arr[i],end=" ")
```

Output :

```
C:\Users\Admin\PycharmProjects\Python_Lab\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Python_Lab\merge.py
Given array :
19 15 20 35 16 27

Sorted array :
15 16 19 20 27 35
Process finished with exit code 0
```

ASSIGNMENT NO:8

Q. To create a stack and perform various operation on stack.

```
class Stack:
    def __init__(self):
        self.stack = []
    def push(self, item):
        self.stack.append(item)
        print(f"Pushed {item} to the stack.")
    def pop(self):
        if not self.is_empty():
            popped_item = self.stack.pop()
            print(f"Popped {popped_item} from the stack.")
        else:
            print("Stack is empty. Cannot pop.")
    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        else:
            print("Stack is empty.")
            return None
    def is_empty(self):
        return len(self.stack) == 0
    def size(self):
        return len(self.stack)

def main():
    stack = Stack()
    stack.push(10)
    stack.push(20)
```

```
stack.push(30)
stack.push(40)

print(f"Top element is: {stack.peek()}")
print(f"Stack size is: {stack.size()}")
stack.pop()
stack.pop()
print(f"Is the stack empty? {stack.is_empty()}")
stack.pop()
stack.pop()
print(f"Is the stack empty now? {stack.is_empty()}")
if __name__ == "__main__":
    main()
```

Output :

```
C:\Users\Admin\PycharmProjects\Python_Lab\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Python_Lab\stack.py
Pushed 10 to the stack.
Pushed 20 to the stack.
Pushed 30 to the stack.
Pushed 40 to the stack.
Top element is: 40
Stack size is: 4
Popped 40 from the stack.
Popped 30 from the stack.
Is the stack empty? False
Popped 20 from the stack.
Popped 10 from the stack.
Is the stack empty now? True

Process finished with exit code 0
|
```

ASSIGNMENT NO:9

Q. To create a stack and perform various operation on queue.

```
class Queue:
    def __init__(self):
        self.queue = []
    def enqueue(self, item):
        self.queue.append(item)
        print(f"Enqueued {item} to the queue.")
    def dequeue(self):
        if not self.is_empty():
            dequeued_item = self.queue.pop(0) # Removes the first element (FIFO)
            print(f"Dequeued {dequeued_item} from the queue.")
        else:
            print("Queue is empty. Cannot dequeue.")
    def front(self):
        if not self.is_empty():
            return self.queue[0]
        else:
            print("Queue is empty.")
            return None
    def is_empty(self):
        return len(self.queue) == 0
    def size(self):
        return len(self.queue)

def main():
    q = Queue()
    q.enqueue(10)
    q.enqueue(20)
    q.enqueue(30)
    q.enqueue(40)
```

```
print(f"Front element is: {q.front()}")
print(f"Queue size is: {q.size()}")
print("")
q.dequeue()
q.dequeue()
print(f"Is the queue empty? {q.is_empty()}")
q.dequeue()
q.dequeue()
print(f"Is the queue empty now? {q.is_empty()}")
if __name__ == "__main__":
    main()
```

Output :

```
C:\Users\Admin\PycharmProjects\Python_Lab\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Python_Lab\queue.py
Enqueued 10 to the queue.
Enqueued 20 to the queue.
Enqueued 30 to the queue.
Enqueued 40 to the queue.
Front element is: 10
Queue size is: 4

Dequeued 10 from the queue.
Dequeued 20 from the queue.
Is the queue empty? False
Dequeued 30 from the queue.
Dequeued 40 from the queue.
Is the queue empty now? True

Process finished with exit code 0
```

ASSIGNMENT NO:10

Q. To create a linked list to perform various operation on linked list.

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
class LinkedList:
```

```
    def __init__(self):
```

```
        self.head = None
```

```
    def insertAtBefin(self, data):
```

```
        new_node = Node(data)
```

```
        new_node.next = self.head
```

```
        self.head = new_node
```

```
    def insertAtIndex(self, data, index):
```

```
        if index == 0:
```

```
            self.insertAtBefin(data)
```

```
            return
```

```
        position = 0
```

```
        current_node = self.head
```

```
        while current_node is not None and position + 1 != index:
```

```
            position += 1
```

```
            current_node = current_node.next
```

```
        if current_node is not None:
```

```
            new_node = Node(data)
```

```
            new_node.next = current_node.next
```

```
            current_node.next = new_node
```

```
        else:
```

```
            print("Index not present")
```

```
    def insertAtEnd(self, data):
```

```
        new_node = Node(data)
```

```
if self.head is None:
    self.head = new_node
    return

current_node = self.head
while current_node.next:
    current_node = current_node.next
current_node.next = new_node

def updateNode(self, val, index):
    current_node = self.head
    position = 0
    while current_node is not None and position != index:
        position += 1
        current_node = current_node.next
    if current_node is not None:
        current_node.data = val
    else:
        print("Index not present")

def remove_first_node(self):
    if self.head is None:
        return
    self.head = self.head.next

def remove_last_node(self):
    if self.head is None:
        return
    if self.head.next is None:
        self.head = None
        return
    current_node = self.head
    while current_node.next and current_node.next.next:
        current_node = current_node.next
    current_node.next = None

def remove_at_index(self, index):
```

```

if self.head is None:
    return

if index == 0:
    self.remove_first_node()
    return

current_node = self.head
position = 0

while current_node is not None and current_node.next is not None and position + 1 != index:
    position += 1
    current_node = current_node.next

if current_node is not None and current_node.next is not None:
    current_node.next = current_node.next.next
else:
    print("Index not present")

def remove_node(self, data):
    current_node = self.head

    if current_node is not None and current_node.data == data:
        self.remove_first_node()
        return

    while current_node is not None and current_node.next is not None:
        if current_node.next.data == data:
            current_node.next = current_node.next.next
            return

        current_node = current_node.next

    print("Node with the five data not found")

def sizeOfLL(self):
    size = 0

    current_node = self.head

    while current_node:
        size += 1
        current_node = current_node.next

    return size

```

```
def printLL(self):
    current_node = self.head
    while current_node:
        print(current_node.data)
        current_node = current_node.next

l1 = LinkedList()
l1.insertAtEnd('a')
l1.insertAtEnd('b')
l1.insertAtBegin('c')
l1.insertAtEnd('d')
l1.insertAtIndex('f', 2)
print("Node Data:")
l1.printLL()
print("\nRemove First Node:")
l1.remove_first_node()
l1.printLL()
print("\nRemove Last Node:")
l1.remove_last_node()
l1.printLL()
print("\nRemove Node at Index 1:")
l1.remove_at_index(1)
l1.printLL()
print("\nLinked list after removing a node:")
l1.printLL()
print("\nUpdate node Value at Index 0:")
l1.updateNode('z', 0)
l1.printLL()
print("\nSize of linked list:", l1.sizeOfLL())
```

Output :

```
C:\Users\Admin\PycharmProjects\Python_Lab\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Python_Lab\linked_list.py
Node Data:
c
a
f
b
d

Remove First Node:
a
f
b
d

Remove Last Node:
a
f
b
```

```
Remove Node at Index 1:
a
b

Linked list after removing a node:
a
b

Update node Value at Index 0:
z
b

Size of linked list: 2

Process finished with exit code 0
```